ML + DL Assignment: Smart Energy Consumption Forecasting & Anomaly Detection System

---

Assignment Overview

Build an end-to-end energy consumption forecasting and anomaly detection system for a simulated smart building environment. This assignment combines classical Machine Learning with Deep Learning techniques and requires comparative analysis and thoughtful engineering decisions.

Dataset: UCI "Appliances Energy Prediction" Dataset

Link: [Appliances Energy Prediction - UCI Machine Learning Repository]

This dataset contains approximately 19,735 instances with features like temperature and humidity readings from sensors in different rooms, weather data, and energy consumption in Wh recorded at 10-minute intervals over approximately 4.5 months.

The same dataset will be used across all four parts of this assignment. In Part B, you will treat it as a tabular regression problem. In Parts C and D, you will treat it as a time-series problem.

PART A — Exploratory Data Analysis and Feature Engineering

1. Exploratory Data Analysis

Perform comprehensive EDA with at least 10 meaningful visualizations. Each visualization must include a written interpretation of 3 to 4 sentences explaining the business or engineering insight it reveals. Generic plots without interpretation will not receive marks.

2. Feature Engineering

Engineer at least 8 new features from the existing data. Examples include but are not limited to:

- Rolling statistics such as rolling mean and rolling standard deviation over different window sizes.
- Time-based features such as hour of day, day of week, is_weekend, and is_peak_hour.
- Lag features such as t-1, t-2, t-6, t-12, t-24, and t-48.
- Interaction features such as the temperature differential between inside and outside.
- Fourier features for capturing cyclical patterns.

Write a 1-page justification document within the notebook explaining your reasoning behind each engineered feature and your overall feature engineering philosophy.

3. Correlation and Mutual Information Analysis

Perform both correlation analysis and mutual information analysis. Compare what each method reveals about the relationship between features and the target variable. Highlight where the two methods agree and where they disagree, and explain why those disagreements might occur.

Deliverable: A clean, well-commented Jupyter notebook with markdown explanations.

PART B — Classical ML Pipeline
Build a regression pipeline to predict energy consumption (the Appliances column).

From-Scratch Implementation (NumPy and Pandas only, no scikit-learn):

- Linear Regression using the Normal Equation.
- Linear Regression using Batch Gradient Descent with learning rate tuning.

For each from-scratch model, show the following:

- Cost function convergence plot (for Gradient Descent).
- Final learned weights.
- Interpretation of the top 5 most influential features based on the weights.
- Training time comparison between the two approaches.

Implement using scikit-learn:

- Random Forest Regressor
- XGBoost or LightGBM (Gradient Boosted Trees)

Hyperparameter Tuning:

- Use Optuna (not GridSearch/RandomSearch) to tune at least 3 models.
- Show optimization history plots and explain your search space choices.

Evaluation:

- Use Time-Series aware cross-validation (not random K-Fold — explain why).
- Metrics: MAE, RMSE, MAPE, and R^2.
- Create a comprehensive comparison table.
- Plot actual vs. predicted values for each model on the test set.
- Plot residual distributions for your top-3 models and analyze if the mathematical assumptions are met.

Feature Importance Analysis:

- Compare feature importances across Random Forest, XGBoost, and Lasso.
- Use SHAP values for the best tree-based model. Include a summary plot and dependence plots for the top-3 features.
- Write a half-page analysis on what the models are actually learning.

Deliverable: A clean, well-commented Jupyter notebook with markdown explanations.

PART C — Deep Learning Pipeline

Treat this as a time-series forecasting problem. Predict the next 6 time steps (1 hour ahead) of energy consumption given the past 48 time steps (8 hours) of multivariate data.

1. Data Preparation

- Create sliding window sequences with an input length of 48 steps and output length of 6 steps.
- Perform a chronological train, validation, and test split. There must be no data leakage. Explain your splitting strategy.
- Apply appropriate scaling. Explain your choice of scaler and why it is suitable for this task.
- Create custom PyTorch Dataset and DataLoader classes. You must use PyTorch for all DL work in this assignment. Do not use Keras or TensorFlow.

2. Model Architectures - Implement the following two architectures:

Model 1: Vanilla LSTM

- At least 2 stacked LSTM layers with dropout.
- Experiment with at least 2 different hidden sizes (for example, 64 and 128).

Model 2: 1D-CNN + LSTM Hybrid

- Use 1D convolutional layers to extract local patterns from the input sequence.
- Feed the CNN output into an LSTM for temporal modeling.
- Experiment with at least 2 different kernel sizes.

3. Training Requirements

- Implement early stopping and learning rate scheduling. Briefly explain your choices.
- Show training and validation loss curves for both models.
- Experiment with at least 2 different loss functions (for example, MSE and Huber Loss) on any one model and compare the results.

4. Evaluation - Report the same metrics as Part B: MAE, RMSE, MAPE, and R-squared on the test set for both models.

Show per-step prediction quality. Demonstrate how the error changes from step 1 to step 6 across the forecast horizon.

Create a visual dashboard using matplotlib subplots that includes:

- At least 2 sample windows showing actual versus predicted values across all 6 forecast steps for each model.
- A chart showing the error distribution per forecast horizon step.

Write a comparison between your best ML model from Part B and your best DL model from Part C. Discuss which performed better and provide possible reasons why.

Deliverable: A clean, well-commented Jupyter notebook with markdown explanations.

PART D — Anomaly Detection Module

Using the same dataset, build an anomaly detection system to flag unusual energy consumption patterns.

D1. Synthetic Anomaly Injection

Inject the following 3 types of anomalies into the test portion of the dataset:

- Point anomalies: Random spikes of 3x to 5x normal consumption at isolated timestamps.
- Contextual anomalies: Normal consumption values placed at unusual times, such as high consumption at 3 AM.
- Collective anomalies: A sustained unusual pattern lasting 1 to 2 hours.

Document exactly how and where you injected the anomalies, including the timestamps and the logic used.

D2. ML-Based Anomaly Detection

Implement the following two methods:

1. Isolation Forest
2. One-Class SVM

Evaluate using Precision, Recall, F1-Score, and PR-AUC. Explain in writing why PR-AUC is more appropriate than ROC-AUC for this task.

D3. DL-Based Anomaly Detection

Build an LSTM Autoencoder in PyTorch:

- The Encoder should compress a sequence of 48 time steps into a compact latent vector.
- The Decoder should reconstruct the original 48 time steps from the latent vector.
- The anomaly score for each window is the reconstruction error.

Determine an appropriate anomaly threshold using the reconstruction error distribution on the validation set.

Plot the reconstruction error over time and overlay the locations of the injected anomalies.

D4. Comparison and Analysis

Create a comparison table showing which method (ML-based or DL-based) performs better for each type of anomaly (point, contextual, collective).

Visualize the detected anomalies on a timeline plot, color-coded by anomaly type.

Write a half-page analysis discussing false positives, false negatives, and what you would consider for deploying such a system in a real building.

Deliverable: A clean, well-commented Jupyter notebook with markdown explanations.